

L06: Network Algorithms

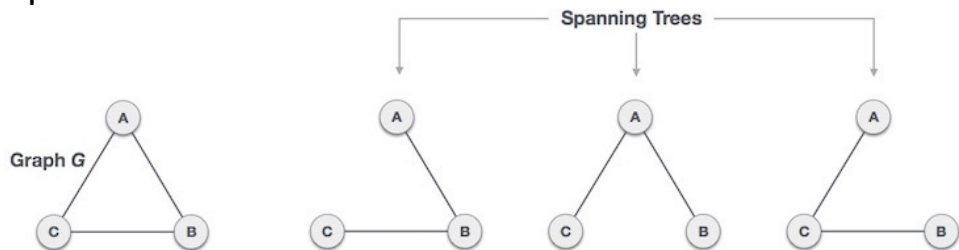
Monika Terentjeviene



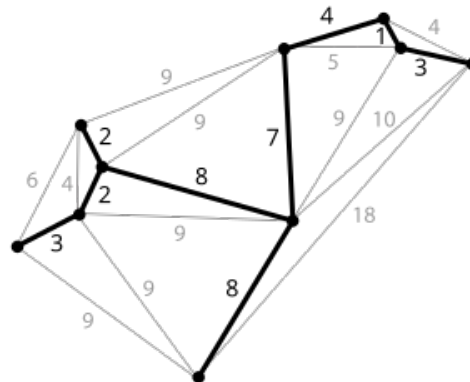
- Spanning Trees
 - Minimum Spanning Tree
 - Kruskal's Algorithm
- Shortest Paths
 - Dijkstra's Algorithm
- Network
 - s-t-flow
 - Ford-Fulkerson Algorithm for Maximum Flows
 - Minimum-cost flow problem

A **spanning tree** of an undirected graph is a subgraph that

- contains all the vertices of the graph,
- is connected,
- is acyclic.



A **minimum spanning tree (MST)** is a spanning tree of an edge-weighted undirected graph with the minimum possible total edge weight.



Kruskal's Algorithm is used to find the minimum spanning tree of an edge-weighted undirected graph.

Input: a connected undirected graph G with edge weights $c: E(G) \rightarrow \mathbb{R}$.

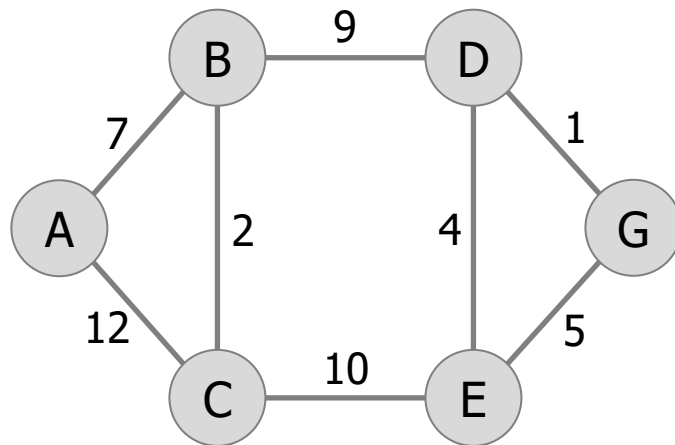
Output: a minimum weight spanning tree $(V(G), T)$ in G .

Sort $E(G) = \{e_1, \dots, e_m\}$ such that $c(e_1) \leq c(e_2) \leq \dots \leq c(e_m)$

$T \leftarrow \emptyset$

for $i \leftarrow 1$ **to** m **do**

if $(V(G), T \cup \{e_i\})$ is a forest **then** $T \leftarrow T \cup \{e_i\}$



Kruskal's Algorithm is used to find the minimum spanning tree of an edge-weighted undirected graph.

Input: a connected undirected graph G with edge weights $c: E(G) \rightarrow \mathbb{R}$.

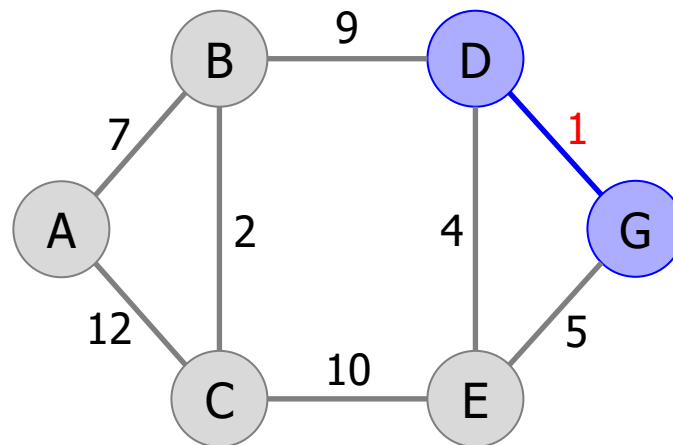
Output: a minimum weight spanning tree $(V(G), T)$ in G .

Sort $E(G) = \{e_1, \dots, e_m\}$ such that $c(e_1) \leq c(e_2) \leq \dots \leq c(e_m)$

$T \leftarrow \emptyset$

for $i \leftarrow 1$ **to** m **do**

if $(V(G), T \cup \{e_i\})$ is a forest **then** $T \leftarrow T \cup \{e_i\}$



Kruskal's Algorithm is used to find the minimum spanning tree of an edge-weighted undirected graph.

Input: a connected undirected graph G with edge weights $c: E(G) \rightarrow \mathbb{R}$.

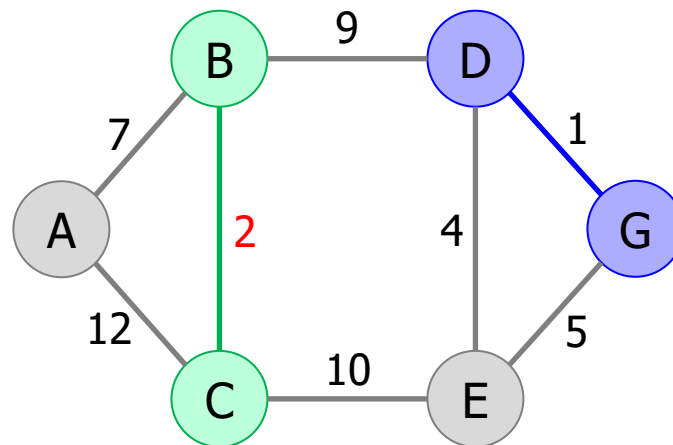
Output: a minimum weight spanning tree $(V(G), T)$ in G .

Sort $E(G) = \{e_1, \dots, e_m\}$ such that $c(e_1) \leq c(e_2) \leq \dots \leq c(e_m)$

$T \leftarrow \emptyset$

for $i \leftarrow 1$ **to** m **do**

if $(V(G), T \cup \{e_i\})$ is a forest **then** $T \leftarrow T \cup \{e_i\}$



Kruskal's Algorithm is used to find the minimum spanning tree of an edge-weighted undirected graph.

Input: a connected undirected graph G with edge weights $c: E(G) \rightarrow \mathbb{R}$.

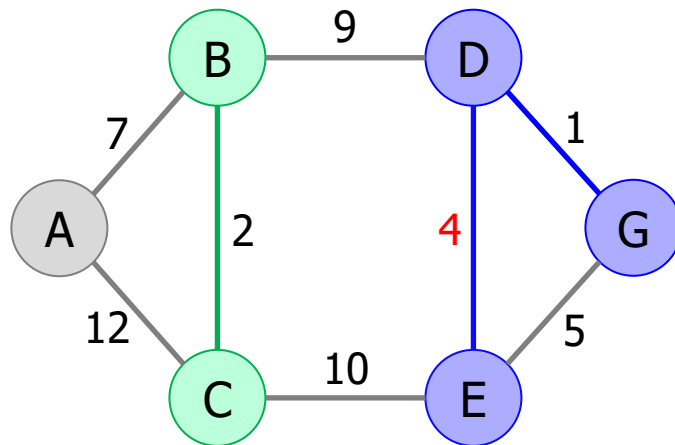
Output: a minimum weight spanning tree $(V(G), T)$ in G .

Sort $E(G) = \{e_1, \dots, e_m\}$ such that $c(e_1) \leq c(e_2) \leq \dots \leq c(e_m)$

$T \leftarrow \emptyset$

for $i \leftarrow 1$ **to** m **do**

if $(V(G), T \cup \{e_i\})$ is a forest **then** $T \leftarrow T \cup \{e_i\}$



Kruskal's Algorithm is used to find the minimum spanning tree of an edge-weighted undirected graph.

Input: a connected undirected graph G with edge weights $c: E(G) \rightarrow \mathbb{R}$.

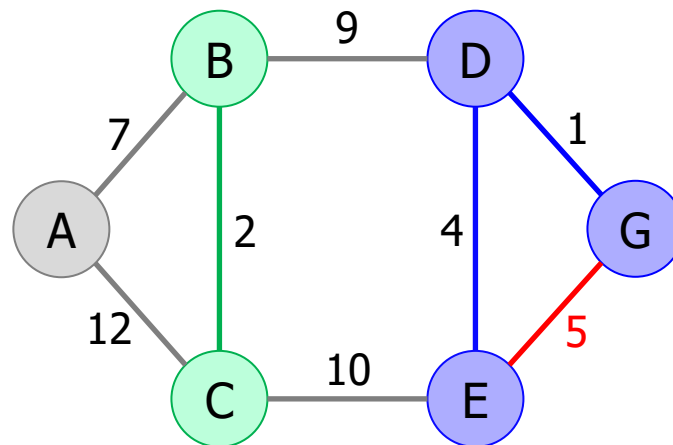
Output: a minimum weight spanning tree $(V(G), T)$ in G .

Sort $E(G) = \{e_1, \dots, e_m\}$ such that $c(e_1) \leq c(e_2) \leq \dots \leq c(e_m)$

$T \leftarrow \emptyset$

for $i \leftarrow 1$ **to** m **do**

if $(V(G), T \cup \{e_i\})$ is a forest **then** $T \leftarrow T \cup \{e_i\}$



Kruskal's Algorithm is used to find the minimum spanning tree of an edge-weighted undirected graph.

Input: a connected undirected graph G with edge weights $c: E(G) \rightarrow \mathbb{R}$.

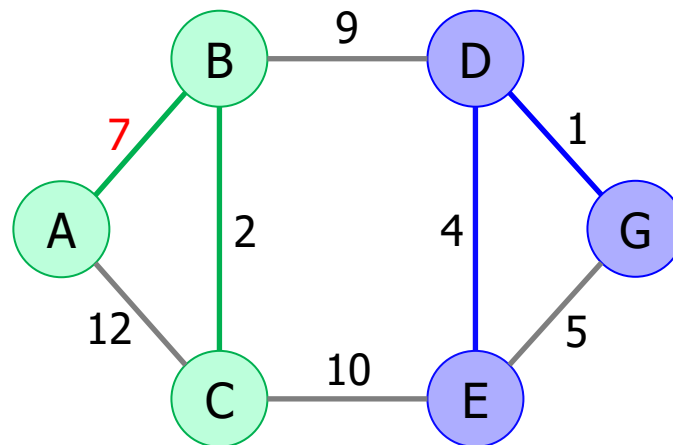
Output: a minimum weight spanning tree $(V(G), T)$ in G .

Sort $E(G) = \{e_1, \dots, e_m\}$ such that $c(e_1) \leq c(e_2) \leq \dots \leq c(e_m)$

$T \leftarrow \emptyset$

for $i \leftarrow 1$ **to** m **do**

if $(V(G), T \cup \{e_i\})$ is a forest **then** $T \leftarrow T \cup \{e_i\}$



Kruskal's Algorithm is used to find the minimum spanning tree of an edge-weighted undirected graph.

Input: a connected undirected graph G with edge weights $c: E(G) \rightarrow \mathbb{R}$.

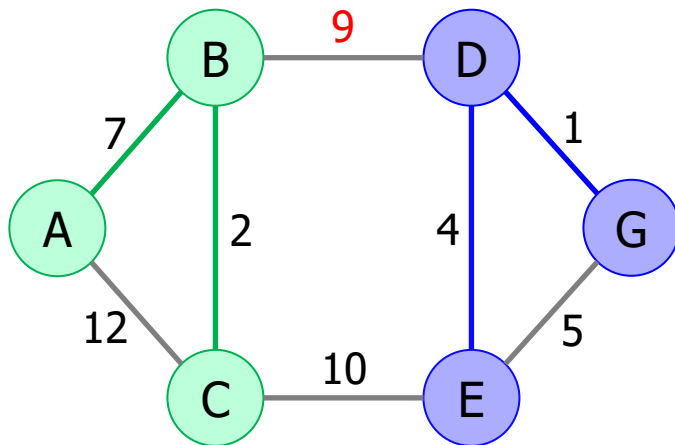
Output: a minimum weight spanning tree $(V(G), T)$ in G .

Sort $E(G) = \{e_1, \dots, e_m\}$ such that $c(e_1) \leq c(e_2) \leq \dots \leq c(e_m)$

$T \leftarrow \emptyset$

for $i \leftarrow 1$ **to** m **do**

if $(V(G), T \cup \{e_i\})$ is a forest **then** $T \leftarrow T \cup \{e_i\}$



Kruskal's Algorithm is used to find the minimum spanning tree of an edge-weighted undirected graph.

Input: a connected undirected graph G with edge weights $c: E(G) \rightarrow \mathbb{R}$.

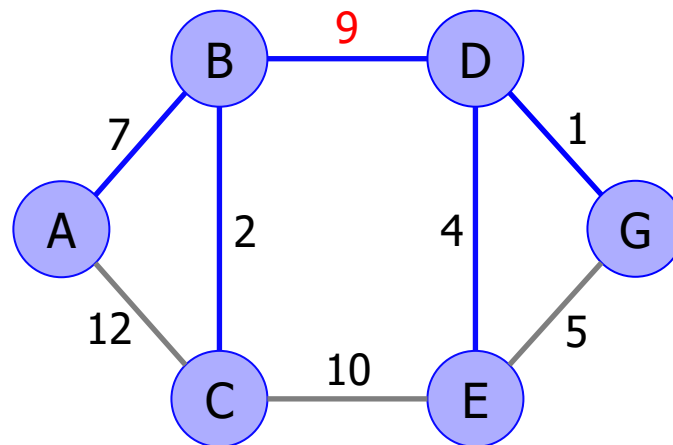
Output: a minimum weight spanning tree $(V(G), T)$ in G .

Sort $E(G) = \{e_1, \dots, e_m\}$ such that $c(e_1) \leq c(e_2) \leq \dots \leq c(e_m)$

$T \leftarrow \emptyset$

for $i \leftarrow 1$ **to** m **do**

if $(V(G), T \cup \{e_i\})$ is a forest **then** $T \leftarrow T \cup \{e_i\}$



Kruskal's Algorithm is used to find the minimum spanning tree of an edge-weighted undirected graph.

Input: a connected undirected graph G with edge weights $c: E(G) \rightarrow \mathbb{R}$.

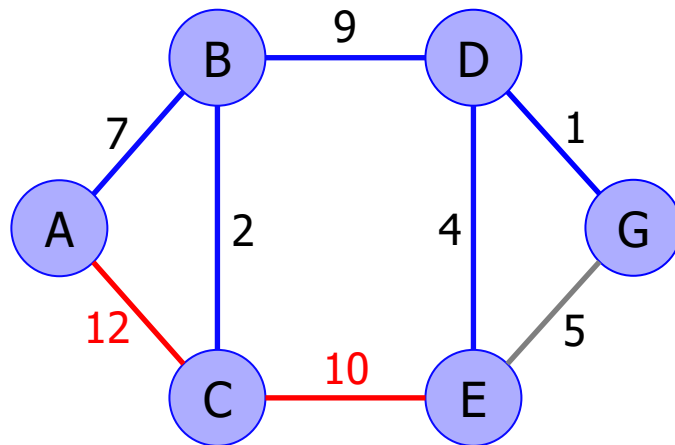
Output: a minimum weight spanning tree $(V(G), T)$ in G .

Sort $E(G) = \{e_1, \dots, e_m\}$ such that $c(e_1) \leq c(e_2) \leq \dots \leq c(e_m)$

$T \leftarrow \emptyset$

for $i \leftarrow 1$ **to** m **do**

if $(V(G), T \cup \{e_i\})$ is a forest **then** $T \leftarrow T \cup \{e_i\}$



Kruskal's Algorithm is used to find the minimum spanning tree of an edge-weighted undirected graph.

Input: a connected undirected graph G with edge weights $c: E(G) \rightarrow \mathbb{R}$.

Output: a minimum weight spanning tree $(V(G), T)$ in G .

Sort $E(G) = \{e_1, \dots, e_m\}$ such that $c(e_1) \leq c(e_2) \leq \dots \leq c(e_m)$

$T \leftarrow \emptyset$

for $i \leftarrow 1$ **to** m **do**

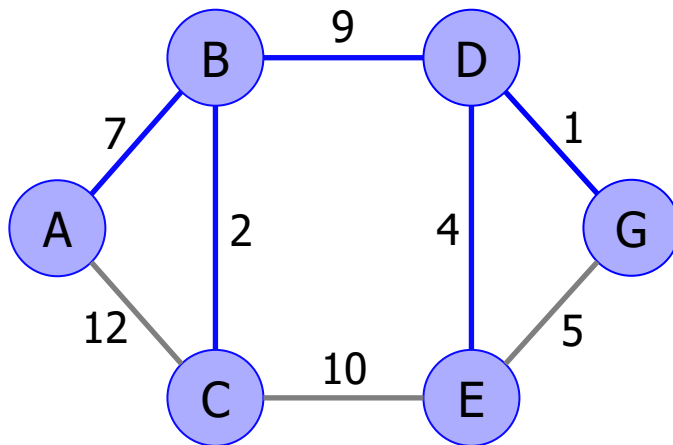
if $(V(G), T \cup \{e_i\})$ is a forest **then** $T \leftarrow T \cup \{e_i\}$

Time complexity is $O(m \log(n))$, where

- m is the number of edges
- n is the number of vertices

Other algorithms:

- Prim's Algorithm



Dijkstra's Algorithm is used to find the **shortest path between two vertices** of an edge-weighted undirected graph.

Input: a digraph G with edge weights $c: E(G) \rightarrow \mathbb{R}_{\geq 0}$, a vertex $s \in V(G)$.

Output: an arborescence $A := (R, T)$ in G such that R contains all vertices reachable from s and $\text{dist}_{(G,c)}(s, v) = \text{dist}_{(A,c)}(s, v)$ holds for all $v \in R$.

$R \leftarrow \emptyset, Q \leftarrow \{s\}, l(s) \leftarrow 0$

while $Q \neq \emptyset$ **do**

 choose a $v \in Q$ with minimum $l(v)$

$Q \leftarrow Q \setminus \{v\}$

$R \leftarrow R \cup \{v\}$

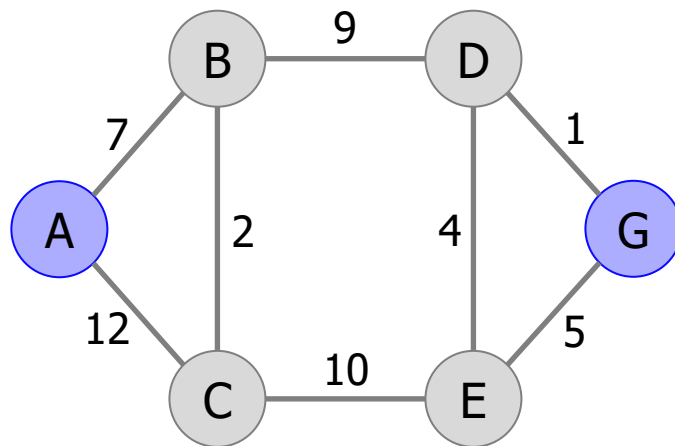
for $e = (v, w) \in \delta_G^+(v)$ with $w \notin R$ **do**

if $w \notin Q$ **or** $l(v) + c(e) < l(w)$

then $l(w) \leftarrow l(v) + c(e), p(w) \leftarrow e$

$Q \leftarrow Q \cup \{w\}$

$T \leftarrow \{p(v) : v \in R \setminus \{s\}\}$



Dijkstra's Algorithm is used to find the **shortest path between two vertices** of an edge-weighted undirected graph.

Input: a digraph G with edge weights $c: E(G) \rightarrow \mathbb{R}_{\geq 0}$, a vertex $s \in V(G)$.

Output: an arborescence $A := (R, T)$ in G such that R contains all vertices reachable from s and $\text{dist}_{(G,c)}(s, v) = \text{dist}_{(A,c)}(s, v)$ holds for all $v \in R$.

$R \leftarrow \emptyset, Q \leftarrow \{s\}, l(s) \leftarrow 0$

while $Q \neq \emptyset$ **do**

 choose a $v \in Q$ with minimum $l(v)$

$Q \leftarrow Q \setminus \{v\}$

$R \leftarrow R \cup \{v\}$

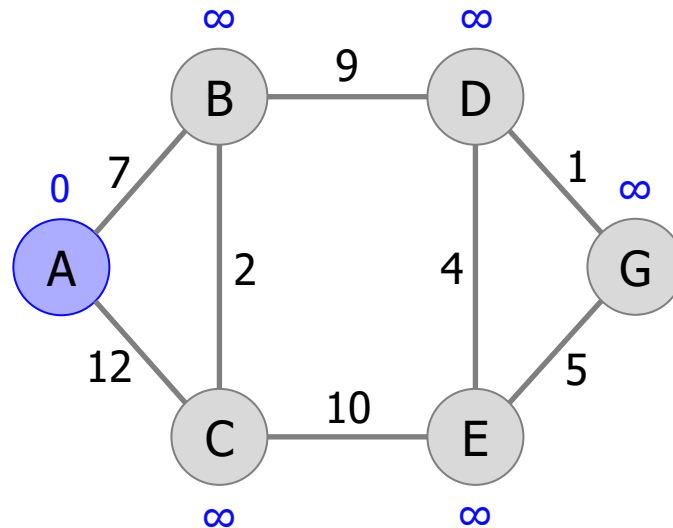
for $e = (v, w) \in \delta_G^+(v)$ with $w \notin R$ **do**

if $w \notin Q$ **or** $l(v) + c(e) < l(w)$

then $l(w) \leftarrow l(v) + c(e), p(w) \leftarrow e$

$Q \leftarrow Q \cup \{w\}$

$T \leftarrow \{p(v) : v \in R \setminus \{s\}\}$



Dijkstra's Algorithm is used to find the **shortest path between two vertices** of an edge-weighted undirected graph.

Input: a digraph G with edge weights $c: E(G) \rightarrow \mathbb{R}_{\geq 0}$, a vertex $s \in V(G)$.

Output: an arborescence $A := (R, T)$ in G such that R contains all vertices reachable from s and $\text{dist}_{(G,c)}(s, v) = \text{dist}_{(A,c)}(s, v)$ holds for all $v \in R$.

$R \leftarrow \emptyset, Q \leftarrow \{s\}, l(s) \leftarrow 0$

while $Q \neq \emptyset$ **do**

 choose a $v \in Q$ with minimum $l(v)$

$Q \leftarrow Q \setminus \{v\}$

$R \leftarrow R \cup \{v\}$

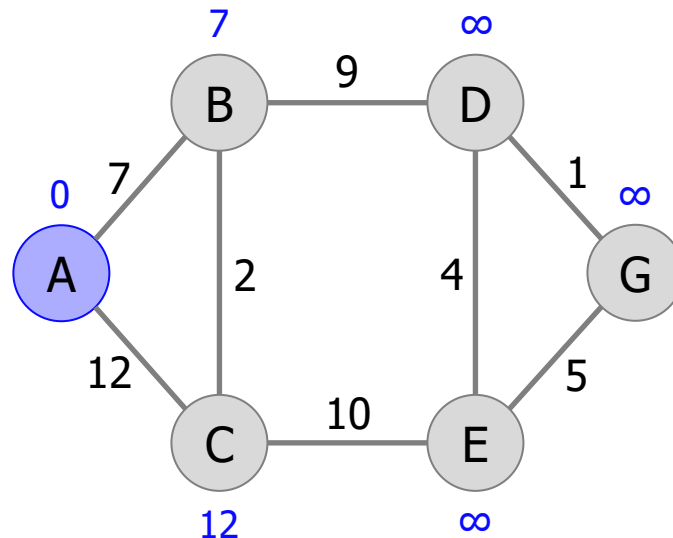
for $e = (v, w) \in \delta_G^+(v)$ with $w \notin R$ **do**

if $w \notin Q$ **or** $l(v) + c(e) < l(w)$

then $l(w) \leftarrow l(v) + c(e), p(w) \leftarrow e$

$Q \leftarrow Q \cup \{w\}$

$T \leftarrow \{p(v) : v \in R \setminus \{s\}\}$



Dijkstra's Algorithm is used to find the **shortest path between two vertices** of an edge-weighted undirected graph.

Input: a digraph G with edge weights $c: E(G) \rightarrow \mathbb{R}_{\geq 0}$, a vertex $s \in V(G)$.

Output: an arborescence $A := (R, T)$ in G such that R contains all vertices reachable from s and $\text{dist}_{(G,c)}(s, v) = \text{dist}_{(A,c)}(s, v)$ holds for all $v \in R$.

$R \leftarrow \emptyset, Q \leftarrow \{s\}, l(s) \leftarrow 0$

while $Q \neq \emptyset$ **do**

 choose a $v \in Q$ with minimum $l(v)$

$Q \leftarrow Q \setminus \{v\}$

$R \leftarrow R \cup \{v\}$

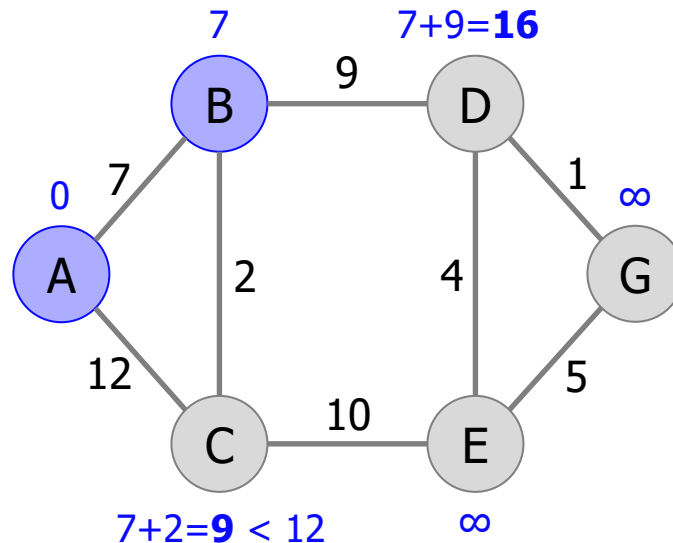
for $e = (v, w) \in \delta_G^+(v)$ with $w \notin R$ **do**

if $w \notin Q$ **or** $l(v) + c(e) < l(w)$

then $l(w) \leftarrow l(v) + c(e), p(w) \leftarrow e$

$Q \leftarrow Q \cup \{w\}$

$T \leftarrow \{p(v) : v \in R \setminus \{s\}\}$



Dijkstra's Algorithm is used to find the **shortest path between two vertices** of an edge-weighted undirected graph.

Input: a digraph G with edge weights $c: E(G) \rightarrow \mathbb{R}_{\geq 0}$, a vertex $s \in V(G)$.

Output: an arborescence $A := (R, T)$ in G such that R contains all vertices reachable from s and $\text{dist}_{(G,c)}(s, v) = \text{dist}_{(A,c)}(s, v)$ holds for all $v \in R$.

$R \leftarrow \emptyset, Q \leftarrow \{s\}, l(s) \leftarrow 0$

while $Q \neq \emptyset$ **do**

 choose a $v \in Q$ with minimum $l(v)$

$Q \leftarrow Q \setminus \{v\}$

$R \leftarrow R \cup \{v\}$

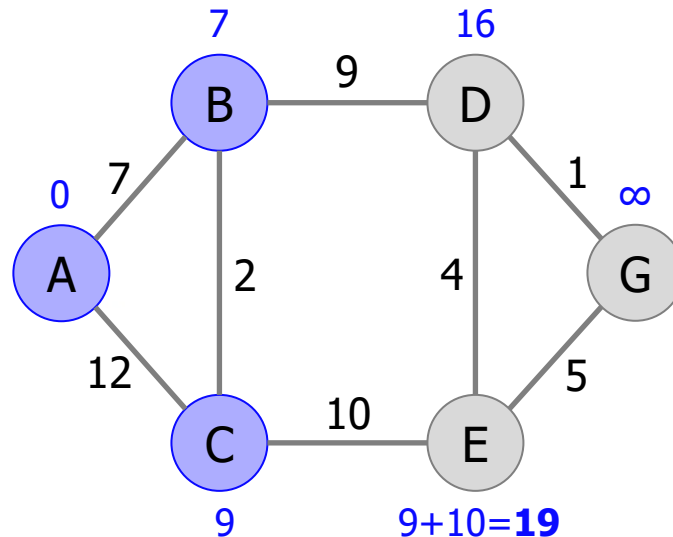
for $e = (v, w) \in \delta_G^+(v)$ with $w \notin R$ **do**

if $w \notin Q$ **or** $l(v) + c(e) < l(w)$

then $l(w) \leftarrow l(v) + c(e), p(w) \leftarrow e$

$Q \leftarrow Q \cup \{w\}$

$T \leftarrow \{p(v) : v \in R \setminus \{s\}\}$



Dijkstra's Algorithm is used to find the **shortest path between two vertices** of an edge-weighted undirected graph.

Input: a digraph G with edge weights $c: E(G) \rightarrow \mathbb{R}_{\geq 0}$, a vertex $s \in V(G)$.

Output: an arborescence $A := (R, T)$ in G such that R contains all vertices reachable from s and $\text{dist}_{(G,c)}(s, v) = \text{dist}_{(A,c)}(s, v)$ holds for all $v \in R$.

$R \leftarrow \emptyset, Q \leftarrow \{s\}, l(s) \leftarrow 0$

while $Q \neq \emptyset$ **do**

 choose a $v \in Q$ with minimum $l(v)$

$Q \leftarrow Q \setminus \{v\}$

$R \leftarrow R \cup \{v\}$

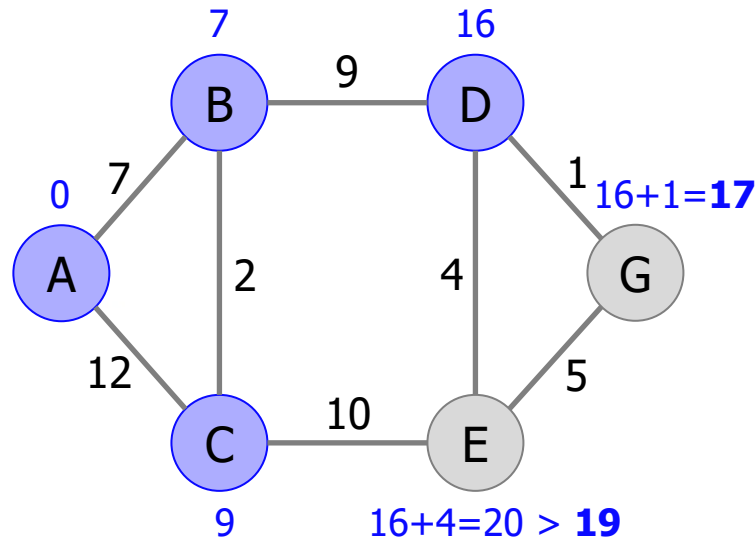
for $e = (v, w) \in \delta_G^+(v)$ with $w \notin R$ **do**

if $w \notin Q$ **or** $l(v) + c(e) < l(w)$

then $l(w) \leftarrow l(v) + c(e), p(w) \leftarrow e$

$Q \leftarrow Q \cup \{w\}$

$T \leftarrow \{p(v) : v \in R \setminus \{s\}\}$



Dijkstra's Algorithm is used to find the **shortest path between two vertices** of an edge-weighted undirected graph.

Input: a digraph G with edge weights $c: E(G) \rightarrow \mathbb{R}_{\geq 0}$, a vertex $s \in V(G)$.

Output: an arborescence $A := (R, T)$ in G such that R contains all vertices reachable from s and $\text{dist}_{(G,c)}(s, v) = \text{dist}_{(A,c)}(s, v)$ holds for all $v \in R$.

$R \leftarrow \emptyset, Q \leftarrow \{s\}, l(s) \leftarrow 0$

while $Q \neq \emptyset$ **do**

 choose a $v \in Q$ with minimum $l(v)$

$Q \leftarrow Q \setminus \{v\}$

$R \leftarrow R \cup \{v\}$

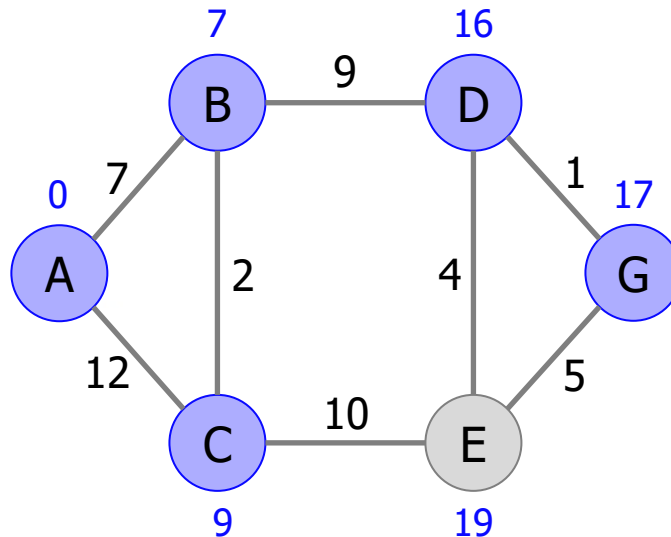
for $e = (v, w) \in \delta_G^+(v)$ with $w \notin R$ **do**

if $w \notin Q$ **or** $l(v) + c(e) < l(w)$

then $l(w) \leftarrow l(v) + c(e), p(w) \leftarrow e$

$Q \leftarrow Q \cup \{w\}$

$T \leftarrow \{p(v) : v \in R \setminus \{s\}\}$



Dijkstra's Algorithm is used to find the **shortest path between two vertices** of an edge-weighted undirected graph.

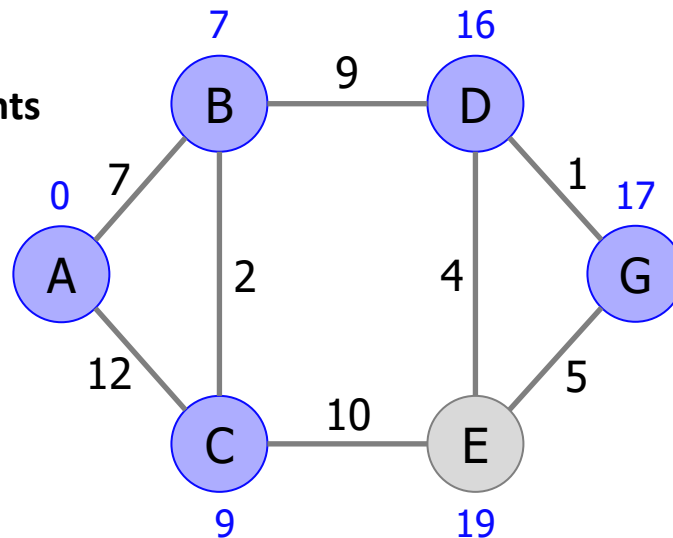
Time complexity is $O(n^2)$, where n is the number of vertices

Important:

- Dijkstra's Algorithm works only for **non-negative weights**

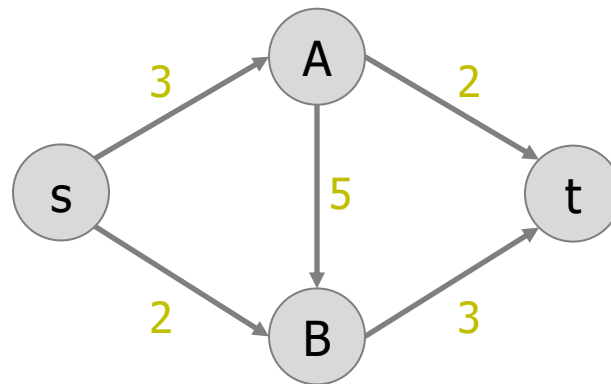
Other algorithms:

- Moore-Bellman-Ford Algorithm



A **network** is a tuple (G, u, s, t) , where

- G is a directed graph
- u is the **capacity** function that maps each edge to $\mathbb{R}_{\geq 0}$
- s is the **source** vertex
- t is the **sink** vertex



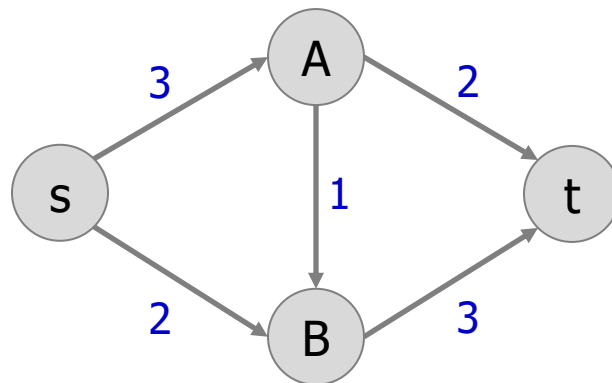
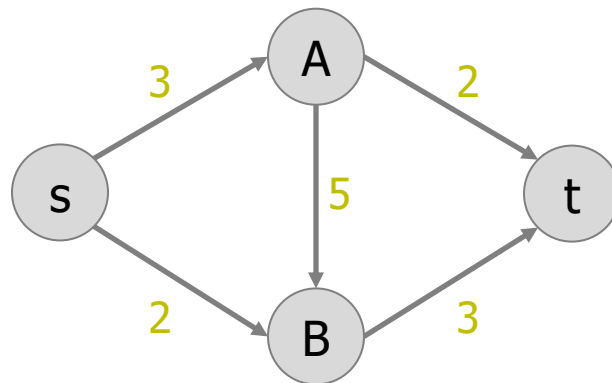
A **network** is a tuple (G, u, s, t) , where

- G is a directed graph
- u is the **capacity** function that maps each edge to $\mathbb{R}_{\geq 0}$
- s is the **source** vertex
- t is the **sink** vertex

An **s-t-flow** is a function f that maps each edge to $\mathbb{R}_{\geq 0}$ with

- $f(e) \leq u(e)$ for all edges e
- For each vertex v with $v \neq s$ and $v \neq t$, the flow going in and out is the same

The **value of flow** is the amount of flow passing from the source to the sink.



Ford-Fulkerson Algorithm is used to find the flow, which **maximizes the value of an s-t-flow**.

Input: a network (G, u, s, t) with integer capacities.

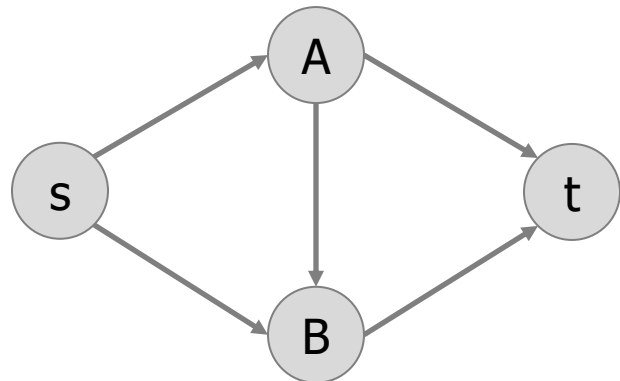
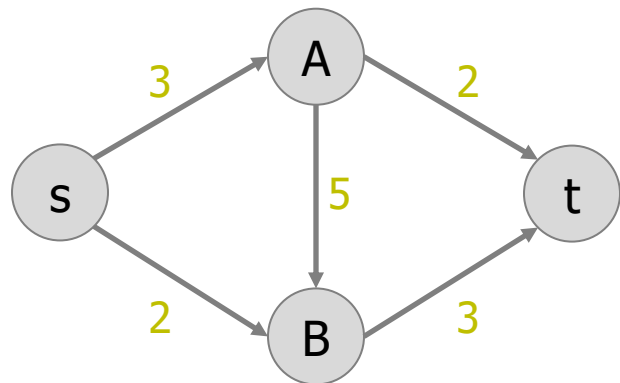
Output: an s - t -flow f of maximum value, with the additional property that the flow is integer-valued.

$f(e) \leftarrow 0$ for all $e \in E(G)$

while there exists an f -augmenting path P **do**

$\gamma \leftarrow \min_{e \in E(P)} u_f(e)$

augment f along P by the amount γ



Ford-Fulkerson Algorithm is used to find the flow, which **maximizes the value of an s-t-flow**.

Input: a network (G, u, s, t) with integer capacities.

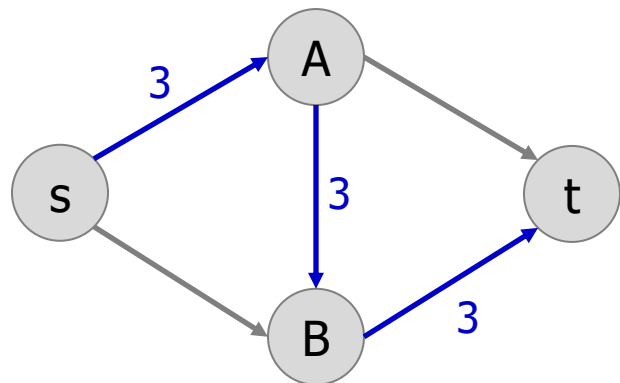
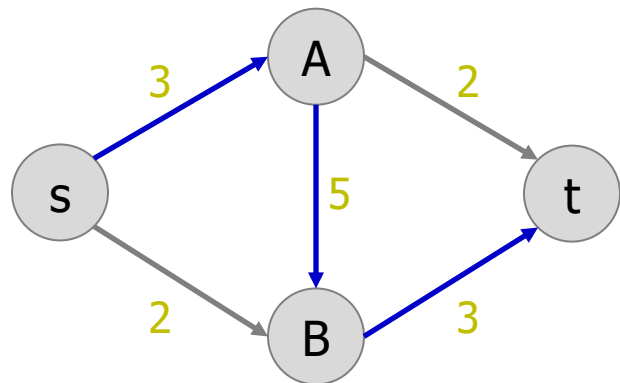
Output: an s - t -flow f of maximum value, with the additional property that the flow is integer-valued.

$f(e) \leftarrow 0$ for all $e \in E(G)$

while there exists an f -augmenting path P **do**

$\gamma \leftarrow \min_{e \in E(P)} u_f(e)$

augment f along P by the amount γ



Ford-Fulkerson Algorithm is used to find the flow, which maximizes the value of an s - t -flow.

Input: a network (G, u, s, t) with integer capacities.

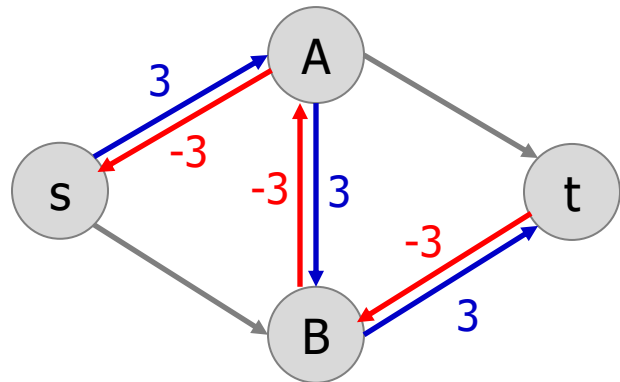
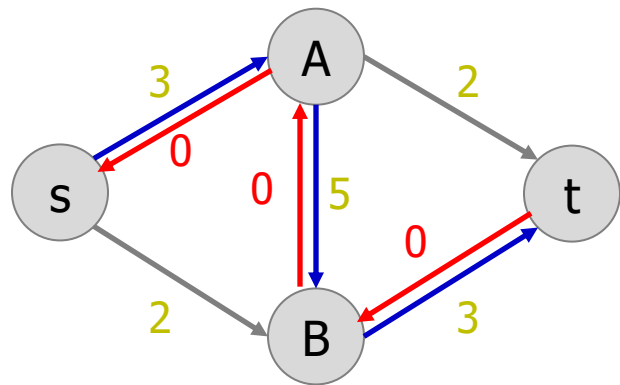
Output: an s - t -flow f of maximum value, with the additional property that the flow is integer-valued.

$f(e) \leftarrow 0$ for all $e \in E(G)$

while there exists an f -augmenting path P **do**

$\gamma \leftarrow \min_{e \in E(P)} u_f(e)$

augment f along P by the amount γ



Ford-Fulkerson Algorithm is used to find the flow, which maximizes the value of an **s-t-flow**.

Input: a network (G, u, s, t) with integer capacities.

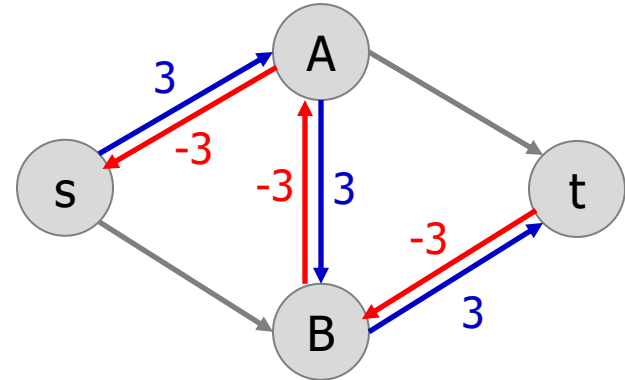
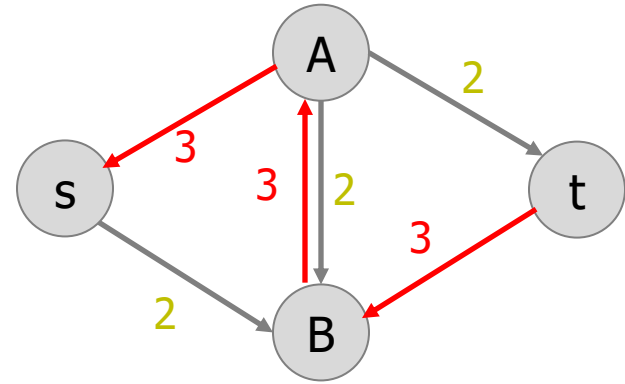
Output: an s - t -flow f of maximum value, with the additional property that the flow is integer-valued.

$f(e) \leftarrow 0$ for all $e \in E(G)$

while there exists an f -augmenting path P **do**

$\gamma \leftarrow \min_{e \in E(P)} u_f(e)$

augment f along P by the amount γ



Ford-Fulkerson Algorithm is used to find the flow, which maximizes the value of an s - t -flow.

Input: a network (G, u, s, t) with integer capacities.

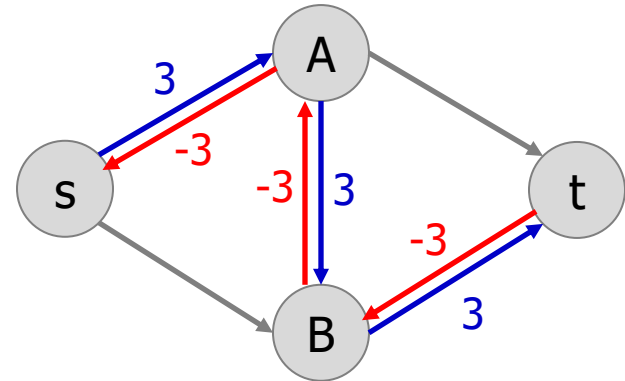
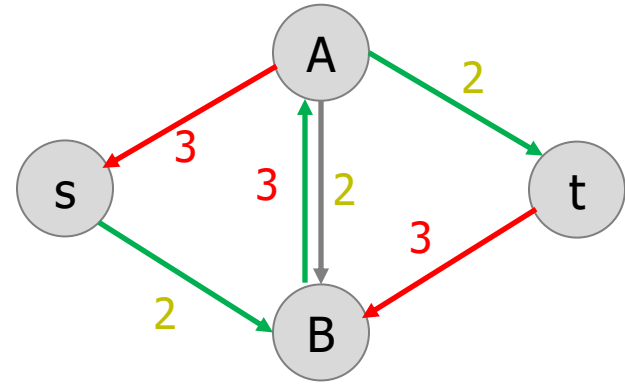
Output: an s - t -flow f of maximum value, with the additional property that the flow is integer-valued.

$f(e) \leftarrow 0$ for all $e \in E(G)$

while there exists an f -augmenting path P **do**

$\gamma \leftarrow \min_{e \in E(P)} u_f(e)$

augment f along P by the amount γ



Ford-Fulkerson Algorithm is used to find the flow, which maximizes the value of an s - t -flow.

Input: a network (G, u, s, t) with integer capacities.

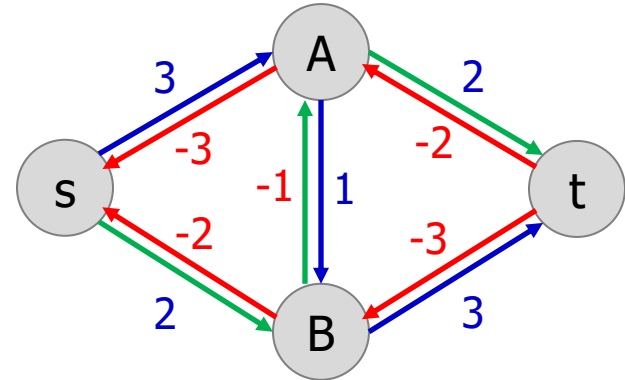
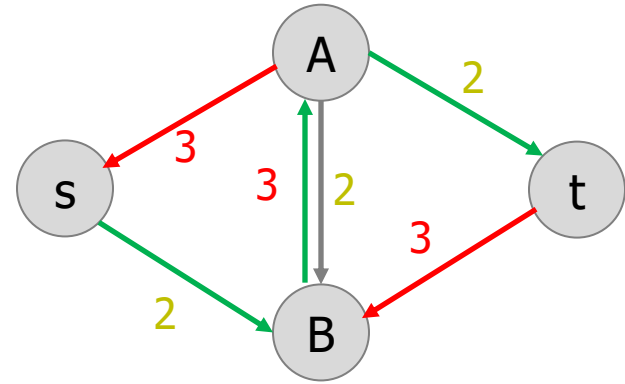
Output: an s - t -flow f of maximum value, with the additional property that the flow is integer-valued.

$f(e) \leftarrow 0$ for all $e \in E(G)$

while there exists an f -augmenting path P **do**

$\gamma \leftarrow \min_{e \in E(P)} u_f(e)$

augment f along P by the amount γ



Ford-Fulkerson Algorithm is used to find the flow, which **maximizes the value of an s-t-flow**.

Input: a network (G, u, s, t) with integer capacities.

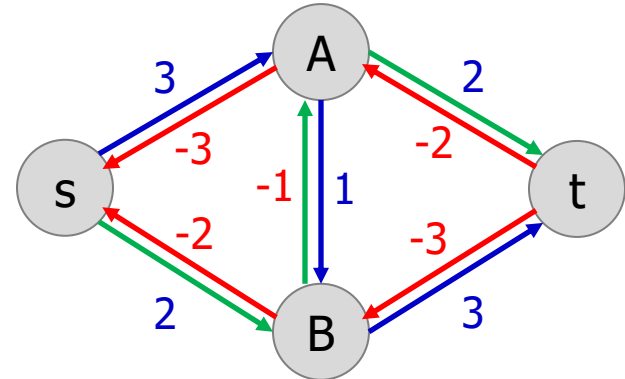
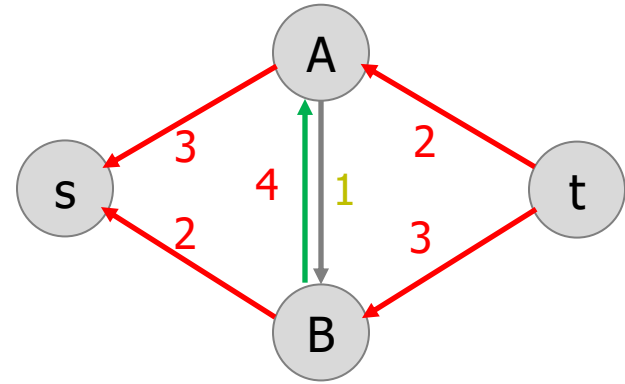
Output: an s - t -flow f of maximum value, with the additional property that the flow is integer-valued.

$f(e) \leftarrow 0$ for all $e \in E(G)$

while there exists an f -augmenting path P **do**

$\gamma \leftarrow \min_{e \in E(P)} u_f(e)$

augment f along P by the amount γ



Ford-Fulkerson Algorithm is used to find the flow, which **maximizes the value of an s-t-flow**.

Input: a network (G, u, s, t) with integer capacities.

Output: an s - t -flow f of maximum value, with the additional property that the flow is integer-valued.

$f(e) \leftarrow 0$ for all $e \in E(G)$

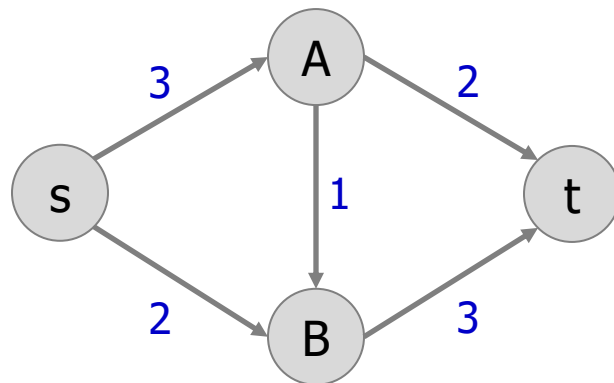
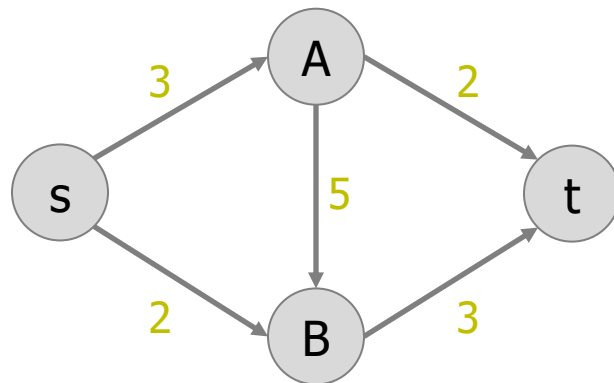
while there exists an f -augmenting path P **do**

$\gamma \leftarrow \min_{e \in E(P)} u_f(e)$

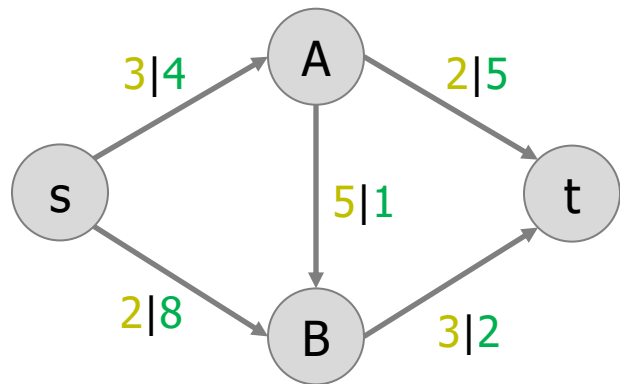
augment f along P by the amount γ

Time complexity can be $O(mW)$, where

- m is the number of edges
- W is the value of a maximum s-t-flow



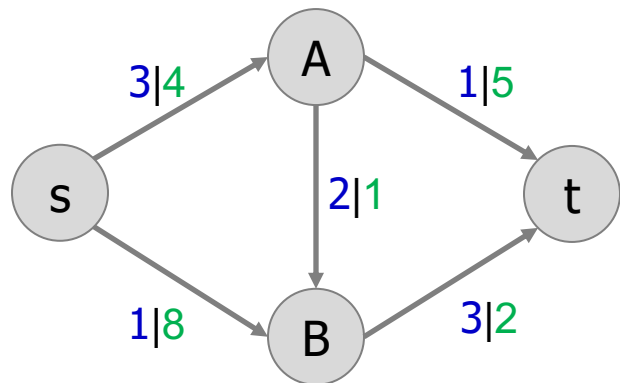
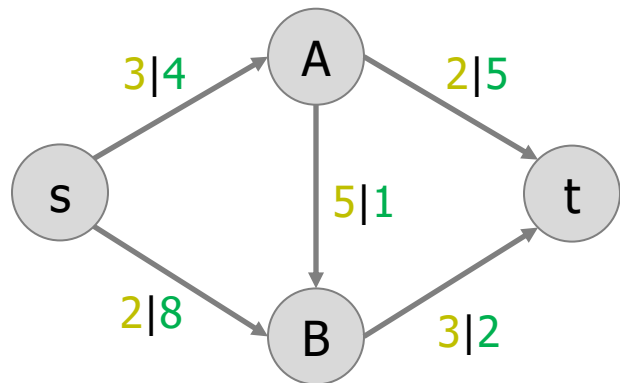
Let us consider a **network** that in addition to **capacity** has a **cost** function c , which maps each edge to $\mathbb{R}_{\geq 0}$.



Let us consider a **network** that in addition to **capacity** has a **cost** function c , which maps each edge to $\mathbb{R}_{\geq 0}$.

The objective is to find an **s-t-flow** that minimizes the cost for a fixed flow value. For example, let $val(f) = 4$, then the minimum cost is

$$cost(f) = 3 \cdot 4 + 1 \cdot 8 + 2 \cdot 1 + 1 \cdot 5 + 3 \cdot 2 = 33$$



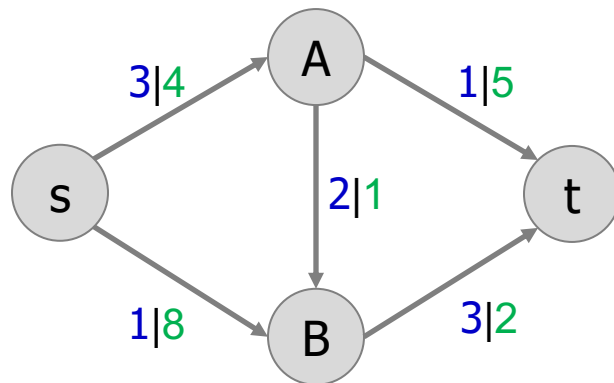
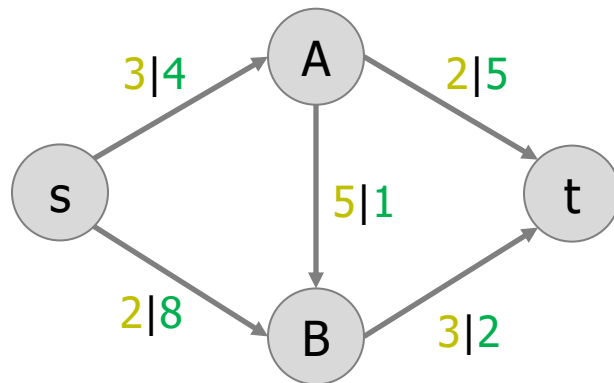
Let us consider a **network** that in addition to **capacity** has a **cost** function c , which maps each edge to $\mathbb{R}_{\geq 0}$.

The objective is to find an **s-t-flow** that minimizes the cost for a fixed flow value. For example, let $val(f) = 4$, then the minimum cost is

$$cost(f) = 3 \cdot 4 + 1 \cdot 8 + 2 \cdot 1 + 1 \cdot 5 + 3 \cdot 2 = 33$$

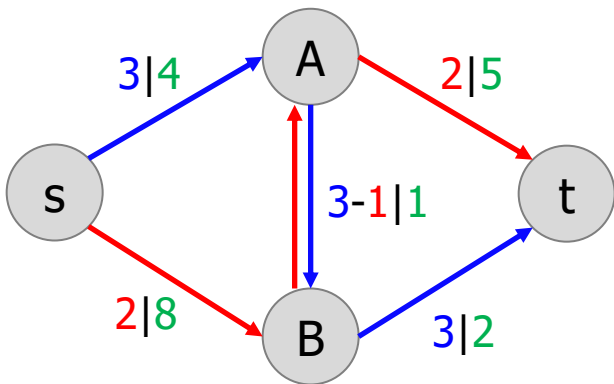
Algorithms:

- Successive Shortest Path (SSP)
- Cycle-Canceling Algorithm



Successive Shortest Path Algorithm

- Find the shortest path from s to t using a shortest path algorithm (e.g. Dijkstra's)
- Send as much flow as possible through the path
- Subtract the flow from the capacities and repeat until the desired flow value is reached



Cycle-Canceling Algorithm

- Send all units of flow through arbitrary paths from s to t
- Construct a residual graph with backward edges (negative costs)
- Cancel all negative valued cycles by changing the flow

